

ADVANTAGE AGGREGATION WITH PRIORITY FOR MULTI-REWARD POLICY OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-reward reinforcement learning (RL) is increasingly used to optimize large language models for both task performance and auxiliary behavioral objectives, such as concise reasoning and efficient tool use. However, existing approaches typically combine objective-specific rewards or advantages symmetrically, implicitly allowing auxiliary objectives to influence the policy update as strongly as the task-defining objective. We term this *advantage priority inversion*: when primary and auxiliary advantages conflict, a sufficiently strong auxiliary signal can reverse the update direction preferred by the primary objective. In light of this, we propose **Primary Advantage Policy Optimization (PAPO)**, a priority-aware framework for multi-reward policy optimization. PAPO explicitly distinguishes primary and auxiliary objectives and performs asymmetric advantage aggregation. Auxiliary advantages are fully preserved when they agree with the primary signal, but are softly bounded under conflict so that they cannot overturn the primary optimization direction. The framework applies naturally to both sequence-level GRPO and token-level PPO; for PPO, we additionally use a multi-head critic to estimate objective-specific returns before advantage fusion. We evaluate PAPO on competition-level mathematical reasoning and agentic tool use with multiple rewards across PPO- and GRPO-based training. PAPO consistently improves primary-task performance while retaining strong auxiliary-objective performance.

1 INTRODUCTION

Reinforcement learning (RL) has become a central component of post-training large language models (LLMs), enabling substantial improvements in reasoning and other complex capabilities (Schulman et al., 2017; Guo et al., 2025). As the post-training objective increases for different tasks, multi-reward policy optimization has become an increasingly important scenario. For example, a reasoning model should answer correctly while avoiding unnecessarily long chain-of-thought (CoT) trajectories; a tool-using agent should complete the task while invoking external tools only when needed (Sun et al., 2026); and an aligned assistant may need to jointly account for helpfulness and safety (Dai et al., 2023). Recent multi-reward methods aggregating multiple rewards into a single scalar objective with independently normalizing heterogeneous rewards or dynamically adapting their weights (Ichihara & Jinnai, 2025; Liu et al., 2026a).

While these approaches combine and balance different reward signals, they largely treat different objectives as symmetric contributors to the final policy update. In many applications, however, the importance of different reward roles is fundamentally asymmetric (e.g., correctness over efficiency). An auxiliary reward objective should therefore refine how a primary goal is achieved, rather than override whether that goal is achieved in the first place. This problem hasn’t been solved very well in multi-reward optimization. For traditional scalar summation methods, after rewards are transformed into advantages, a sufficiently strong auxiliary signal can dominate a conflicting primary signal and reverse the resulting policy-update direction. We refer to this failure mode as *advantage priority inversion*; Independently normalizing different reward dimensions makes signals comparable in scale, but does not encode which objective should determine the optimization direction; Reducing the weight of auxiliary rewards weakens useful supervision even when the objectives agree; Moreover, conditioning auxiliary rewards according to primary rewards still wastes too much supervision on rollouts, causing suboptimal training. These observations suggest that multi-reward optimization

should prioritize one reward direction while still utilizing as many other auxiliary reward signals as possible during reinforcement learning.

To address this problem, we propose **Primary Advantage Policy Optimization (PAPO)**, a general framework for priority-aware multi-reward reinforcement learning. PAPO explicitly distinguishes primary objectives from auxiliary objectives and composes their advantages asymmetrically. When the primary and auxiliary advantage has same direction (sign), PAPO preserves the auxiliary contribution. When they conflict, PAPO softly bounds the auxiliary influence so that it cannot overturn the update direction preferred by the primary objective. In this way, PAPO retains useful auxiliary supervision while preserving the semantic hierarchy among optimization objectives.

Unlike previous methods that only applied to GRPO, PAPO is a general approach that can be used for different RL methods, and we mainly test it on the most popular two RL families. For GRPO-style optimization (Guo et al., 2025), PAPO operates on independently normalized sequence-level advantages before the final policy update. For actor-critic PPO (Schulman et al., 2017), where advantages are estimated at the token level, PAPO performs token-level fusion and introduces a multi-head critic that separately estimates objective-specific returns. This design decouples value estimation from nonlinear advantage aggregation and allows PAPO to handle fine-grained conflicts that may occur within a trajectory.

We evaluate PAPO in two settings with natural primary-auxiliary reward structure: competition-level mathematical reasoning, where answer correctness is primary and reasoning efficiency is auxiliary, and agentic tool use, where task correctness is primary and tool-use efficiency is auxiliary. Across both PPO- and GRPO-based training, PAPO consistently provides stronger primary-task performance while maintaining competitive or better auxiliary-objective behavior. These results demonstrate that explicitly preserving objective priority offers a simple and effective way to treat conflicting rewards. Our contributions are threefold:

- We explore and solve *advantage priority inversion*, a failure mode in multi-reward policy optimization where auxiliary objectives can override the policy-update direction preferred by a primary objective.
- We propose **PAPO**, a priority-aware advantage aggregation framework that preserves primary reward direction while retaining useful auxiliary reward supervision, with generality for both sequence-level GRPO and token-level PPO.
- We validate PAPO on mathematical reasoning and agentic tool use, showing consistent gains in primary-task performance across both PPO- and GRPO-based optimization while preserving strong auxiliary-objective behavior.

2 RELATED WORK

Policy Optimization for LLM Reasoning. Reinforcement learning has become an effective paradigm for improving the reasoning capabilities of large language models. PPO (Schulman et al., 2017) has been widely used for language-model post-training, while GRPO (Guo et al., 2025) eliminates the learned critic by estimating relative advantages from groups of sampled responses. Recent methods further improve the stability and efficiency of GRPO-style optimization. DAPO (Yu et al., 2025) introduces asymmetric clipping, dynamic sampling, and token-level optimization for large-scale reasoning training, while GSPO (Zheng et al., 2025) performs importance weighting and clipping at the sequence level.

Multi-Reward Reinforcement Learning. Multi-reward RL has been increasingly used to optimize multiple dimensions of LLM behavior. Safe RLHF (Dai et al., 2023) separates helpfulness and safety into reward and cost objectives under constrained optimization. Liu et al. (2026b; 2025) condition the length reward on the accuracy reward to increase the efficiency for reasoning. More recently, AW-GRPO (Ichihara & Jinnai, 2025) dynamically adjusts reward weights according to the learning progress of individual objectives, while GDPO (Liu et al., 2026a) identifies signal collapse caused by normalizing aggregated rewards in GRPO and instead normalizes each reward independently before combining their advantages. SMOPD (Wang et al., 2026) further studies imbalanced optimization among heterogeneous rewards and learns reward-specialized policies.

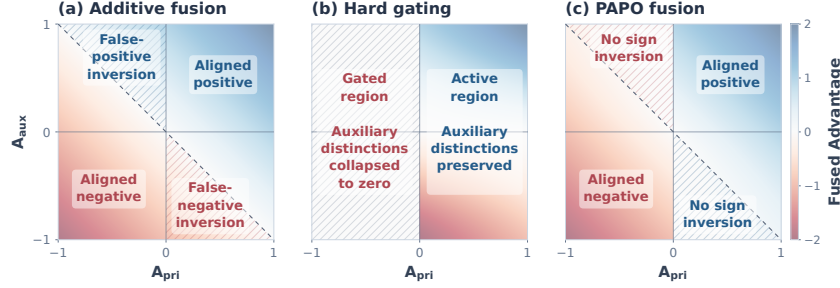


Figure 1: **Aggregated advantage under different primary and auxiliary advantages.** (a) Additive aggregation can reverse the primary update direction under conflicting auxiliary signals. (b) Hard gating avoids such conflicts but discards informative auxiliary distinctions. (c) PAPO preserves auxiliary information while maintaining the direction induced by the primary advantage.

3 PRELIMINARIES

Proximal Policy Optimization. For a token action a_t at state s_t , PPO defines the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and optimizes the clipped surrogate objective

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon_p, 1 + \epsilon_p) \hat{A}_t \right) \right], \quad (1)$$

where ϵ_p is the PPO clipping range and $\hat{A}_t$ is the estimated advantage. In actor-critic PPO, $\hat{A}_t$ is commonly estimated using Generalized Advantage Estimation (GAE). Given a value function V_ϕ , the TD error and GAE are computed as $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ and $\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$, respectively. The value network is trained to regress toward the return target $R_t = \hat{A}_t + V_{\text{old}}(s_t)$, often with a clipped value loss for additional stability.

Group Relative Policy Optimization GRPO is a sequence-level policy optimization method designed for response-level reward feedback. Given a prompt x , GRPO samples a group of responses $\{y_i\}_{i=1}^G$ from the old policy and computes a reward R_i for each response. GRPO estimates the relative advantage of each response by normalizing its reward within the sampled group: $A_i = \frac{R_i - \mu_{\text{group}}}{\sigma_{\text{group}} + \epsilon}$, where μ_{group} and σ_{group} are the mean and standard deviation of rewards within the group. The resulting normalized advantage is then used in a PPO-style clipped objective at the sequence or token level. Let $\mathcal{K}_{i,t} = D_{\text{KL}}(\pi_\theta(\cdot | s_{i,t}) \| \pi_{\text{ref}}(\cdot | s_{i,t}))$. GRPO optimizes

$$L^{\text{GRPO}}(\theta) = \mathbb{E}_{i,t} [\min(r_{i,t}(\theta) A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_p, 1 + \epsilon_p) A_i) - \beta_{\text{KL}} \mathcal{K}_{i,t}], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(a_{i,t}|s_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_{i,t})}$, π_{ref} is a reference policy, and β_{KL} controls the KL penalty.

4 REVISITING MULTI-REWARDS: ADVANTAGE PRIORITY INVERSION

Here we illustrate the limitations of conventional advantage aggregation and hard gating, as well as the motivation behind our priority-aware formulation with Figure 1 (GRPO as an example for aggregated advantages in a group).

Additive aggregation reverses priority. Under common additive aggregation (Guo et al., 2025; Aggarwal & Welleck, 2025), the combined advantage is $A_i^{\text{sum}} = A_{i,\text{pri}} + \alpha A_{i,\text{aux}}$, where $\alpha > 0$ controls the auxiliary contribution. As illustrated in Figure 1(a), when the two advantages conflict and the auxiliary signal has a larger magnitude, i.e., $A_{i,\text{pri}} A_{i,\text{aux}} < 0$ and $\alpha |A_{i,\text{aux}}| > |A_{i,\text{pri}}|$, the aggregated advantage satisfies $\text{sign}(A_i^{\text{sum}}) \neq \text{sign}(A_{i,\text{pri}})$. We refer to this failure mode as *advantage priority inversion*. A response with a negative primary advantage may be incorrectly reinforced because of a large positive auxiliary advantage; conversely, a response supported by the primary objective may be suppressed by a sufficiently negative auxiliary signal. Liu et al. (2026a) tried to adjust the weight of auxiliary advantage, but observes that it’s still very hard to prioritize advantage because the model is optimized according to the advantage which is easier to optimize.

Existing priority mechanisms remain coarse. Previous strategies (Liu et al., 2026a;b; 2025; Team et al., 2025) try to solve priority inversion by gating the auxiliary reward according to the primary

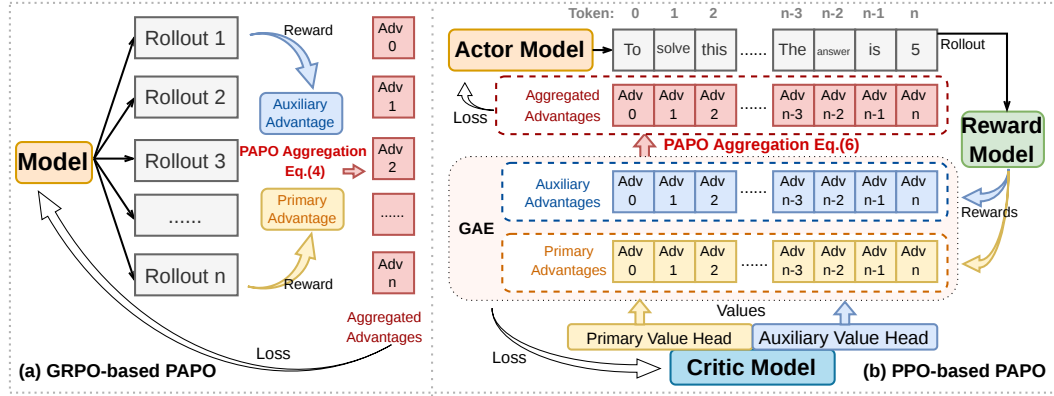


Figure 2: **PAPO overview.** (a) **GRPO-based PAPO.** Given multiple rollouts, primary and auxiliary rewards are converted to sequence-level advantages and combined. (b) **PPO-based PAPO.** A multi-head critic estimates the values for primary and auxiliary objectives. Token-level advantages are then computed using GAE and fused by PAPO before being used to optimize the actor.

reward, e.g., $\tilde{R}_{i,aux} = R_{i,aux} \mathbf{1}\{R_{i,pri} \geq \tau\}$. However, this reward-level gating provides only binary, coarse control. As illustrated in Figure 1(b), hard gating removes auxiliary distinctions among samples that fail the primary advantage condition, discarding all negative samples based on primary rewards and losing a lot of useful supervision signal for distinct auxiliary rewards. Also, once the condition is satisfied, the large auxiliary advantages is restored and may still dominate the primary advantage, causing priority inversion.

Response-level aggregation is insufficient for token-level PPO. Most existing multi-reward methods mentioned in Section 2 typically aggregate objectives at the response level for GRPO-based methods, while they can't be adapted to PPO properly, which can estimate a distinct advantage at every token through GAE. The direction and magnitude of primary and auxiliary advantages may vary across tokens within the same response: even a successful trajectory may contain both useful and harmful actions. Consequently, sequence-level agreement $A_{i,pri}^{seq} A_{i,aux}^{seq} \geq 0$ does not imply token-level agreement, $A_{i,t}^{pri} A_{i,t}^{aux} \geq 0$ for all t . Response-level aggregation therefore cannot resolve fine-grained primary-auxiliary conflicts that arise within a trajectory.

5 METHOD

In light of the limitations analyzed in Section 4, we propose **Primary Advantage Policy Optimization (PAPO)** as illustrated in Figure 2. PAPO explicitly preserves the optimization direction determined by the primary advantage, while still allowing auxiliary advantages to provide adjusted supplementary supervision. PAPO can be applied in different mainstream policy optimization frameworks by operating on advantages, including both sequence-level GRPO, token-level PPO, and their variants with adjustments to advantage estimation and tailored design for the critic model.

5.1 PRIMARY ADVANTAGE POLICY OPTIMIZATION (PAPO)

PAPO supports multiple reward signals on both sides. Let $\mathcal{P}$ denote the set of primary objectives and $\mathcal{Q}$ denote the set of auxiliary objectives. For objective-specific advantages $\{A_m\}_{m \in \mathcal{P} \cup \mathcal{Q}}$, we first aggregate them according to their roles:

$$A_{pri} = \sum_{p \in \mathcal{P}} A_p, \quad A_{aux} = \sum_{q \in \mathcal{Q}} A_q. \quad (3)$$

Optional objective-specific weights can be incorporated into either sum. We then apply a directional asymmetric squashing mechanism:

$$A_{agg} = A_{pri} + \mathbf{1}\{A_{pri} A_{aux} \geq 0\} \alpha A_{aux} + \mathbf{1}\{A_{pri} A_{aux} < 0\} A_{pri} \tanh\left(\frac{\alpha A_{aux}}{A_{pri} + \epsilon \operatorname{sgn}(A_{pri})}\right). \quad (4)$$

Here, α controls the strength of auxiliary supervision and ϵ is a small constant for numerical stability.

When the primary and auxiliary advantages have the same sign, PAPO preserves the auxiliary signal without attenuation, since both objectives favor the same policy-update direction. When they conflict, PAPO uses $\tanh$ to smoothly bound the auxiliary influence relative to the magnitude of the primary advantage. Ignoring the small numerical constant, the conflicting case can be written as $A_{\text{agg}} = A_{\text{pri}} \left(1 - \tanh\left(\frac{\alpha|A_{\text{aux}}|}{|A_{\text{pri}}|}\right)\right)$. Since $\tanh(\cdot) \in [0, 1)$ for nonnegative inputs, the fused advantage always retains the sign of A_{pri} while its magnitude is smoothly attenuated as the auxiliary conflict becomes stronger.

The $\tanh$ transformation provides three desirable properties. First, it is *bounded*: even an arbitrarily large conflicting auxiliary advantage can only reduce the primary update toward zero, rather than reversing its direction. Second, it is *locally linear*: for weak conflicts, $\tanh(x) \approx x$, so PAPO closely recovers standard additive aggregation and preserves fine-grained auxiliary information. Third, the ratio between auxiliary and primary advantages makes the attenuation *scale-adaptive*: the same auxiliary signal has little effect when strongly contradicted by the primary objective, but can substantially modulate the update when the primary evidence is weak. Compared with hard clipping or gating, the smooth saturation of $\tanh$ therefore provides continuous control over auxiliary influence without discarding conflicting supervision.

5.2 GRPO-BASED PAPO: SEQUENCE-LEVEL ADVANTAGE AGGREGATION

When applied to sequence-level optimization frameworks such as Group Relative Policy Optimization (GRPO), PAPO operates directly on the outcome of the generated rollouts. For each generated response, we first compute objective-specific sequence-level rewards or raw advantages $\{A_{i,m}\}_{m \in \mathcal{P} \cup \mathcal{Q}}$. To put heterogeneous reward signals on a comparable scale, each objective-specific signal is independently normalized within the sampled group as

$$A_{i,m} = \frac{A_{i,m} - \mu_{\text{group}}^m}{\sigma_{\text{group}}^m + \epsilon}, \quad m \in \mathcal{P} \cup \mathcal{Q}.$$

We then aggregate the normalized objective-specific advantages according to Eq. 3, obtaining $A_{i,\text{pri}}$ and $A_{i,\text{aux}}$, and fuse them with the asymmetric squashing rule to obtain A_i^{agg} . To maintain the zero-mean baseline property critical for variance reduction after fusion, we further normalize the fused advantage within each sampled group as $A_i = \frac{A_i^{\text{agg}} - \mu_{\text{group}}^{\text{agg}}}{\sigma_{\text{group}}^{\text{agg}} + \epsilon}$. This post-hoc group normalization guarantees that the relative ranking established by the asymmetric squashing is preserved without introducing global mean shifts into the policy gradients.

5.3 PPO-BASED PAPO: TOKEN-LEVEL ADVANTAGE AGGREGATION

Token-Level Advantage with PAPO. Unlike GRPO, which typically assigns a sequence-level advantage to an entire response, actor-critic PPO estimates a distinct advantage at each token through Generalized Advantage Estimation (GAE). This finer granularity is important for multi-reward optimization: even within the same successful trajectory, the primary and auxiliary objectives may agree at some positions but conflict at others. A response-level aggregation can therefore mask local conflicts that directly determine token-level policy updates. We consequently apply PAPO at the same granularity as the PPO actor update.

To retain objective-specific credit assignment, we maintain one value head for each reward objective, forming a multi-head critic $\{V_{\phi}^m(s_t)\}_{m \in \mathcal{M}}$, where $\mathcal{M} = \mathcal{P} \cup \mathcal{Q}$. For each objective m and token step t , we independently compute the TD residual and GAE:

$$\delta_t^m = r_t^m + \gamma V_{\phi}^m(s_{t+1}) - V_{\phi}^m(s_t), \quad \hat{A}_t^m = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}^m. \quad (5)$$

Computing GAE independently before fusion is deliberate. Different rewards may induce substantially different temporal credit-assignment patterns; early scalarization would force a single critic to explain their mixture and obscure which objective is responsible for a particular token-level update. Instead, PAPO treats nonlinear advantage fusion as a *policy-side* operation while leaving each critic head to estimate an ordinary objective-specific return. We then aggregate the token-level advantages

according to their roles, following Eq. 3, yielding $\hat{A}_t^{\text{pri}}$ and $\hat{A}_t^{\text{aux}}$. PAPO is applied independently at every token:

$$\hat{A}_t^{\text{agg}} = \hat{A}_t^{\text{pri}} + \mathbf{1}\left\{\hat{A}_t^{\text{pri}} \hat{A}_t^{\text{aux}} \geq 0\right\} \alpha \hat{A}_t^{\text{aux}} + \mathbf{1}\left\{\hat{A}_t^{\text{pri}} \hat{A}_t^{\text{aux}} < 0\right\} \hat{A}_t^{\text{pri}} \tanh\left(\frac{\alpha \hat{A}_t^{\text{aux}}}{\hat{A}_t^{\text{pri}} + \epsilon \operatorname{sgn}(\hat{A}_t^{\text{pri}})}\right). \quad (6)$$

This token-wise formulation allows the auxiliary objective to reinforce locally compatible actions while attenuating locally conflicting ones, rather than making a single eligibility decision for the entire trajectory. In particular, a globally successful response can still contain inefficient or undesirable intermediate actions, for which token-level PAPO provides finer-grained control.

Multi-Head Critic Training. The critic is intentionally kept separate from PAPO fusion, and we design separate value heads for each reward prediction. Each value head regresses toward the unfused return associated with its own reward objective:

$$R_t^m = \hat{A}_t^m + V_{\text{old}}^m(s_t), \quad m \in \mathcal{M}. \quad (7)$$

This design avoids letting the critic to fit an aggregated target produced by the nonlinear PAPO transformation. If using such a fused target, it would make value estimation unnecessarily coupled to the actor-side aggregation rule and harder to predict accurately. By keeping objective-specific value targets intact, PAPO preserves clear semantics for each critic head and allows the actor to change how objectives are combined without redefining the underlying value functions. Each critic head is optimized independently using the standard clipped value loss:

$$L_m^{\text{VF}}(\phi) = \frac{1}{B} \sum_{\text{batch}} \max\left[\left(V_\phi^m(s_t) - R_t^m\right)^2, \left(\text{clip}\left(V_\phi^m(s_t), V_{\text{old}}^m(s_t) - \epsilon_c, V_{\text{old}}^m(s_t) + \epsilon_c\right) - R_t^m\right)^2\right], \quad (8)$$

where ϵ_c denotes value-clipping range. The final critic objective is $L^{\text{VF}}(\phi) = \sum_{m \in \mathcal{M}} L_m^{\text{VF}}(\phi)$.

This separation yields a useful division of responsibility: the multi-head critic provides objective-specific temporal credit assignment, while PAPO determines how these signals should interact during policy optimization. Thus, objective priority is enforced only where it is needed in the actor update without distorting the value estimation of the underlying reward signals.

6 EXPERIMENTS

We organize our experiments around four research questions: whether PAPO improves the primary objective while retaining useful auxiliary behavior (**RQ1**), how priority-aware aggregation changes the optimization dynamics of competing objectives (**RQ2**), which design components are responsible for the improvement (**RQ3**), and whether the resulting gains generalize across models, scales, reward semantics, and domains (**RQ4**). To answer **RQ1**, we evaluate PAPO on two representative multi-reward settings, competition math reasoning and tool use.

Baselines. For baselines, we mainly include the **Base** model Qwen3-8B (Yang et al., 2025) and standard **PPO** (Schulman et al., 2017) and **GRPO** (Guo et al., 2025) with additive multi-reward aggregation, covering token-level and sequence-level policy optimization. We additionally compare with **GDPO** (Liu et al., 2026a), which separately normalizes objective-specific rewards. Our PAPO shares the same settings as the corresponding PPO or GRPO baseline. We also include additional models and baselines in Section 6.5 and Appendix A to evaluate the effectiveness of PAPO.

6.1 COMPETITION-LEVEL MATH REASONING

Data and Reward Settings. We evaluate PAPO on competition-level mathematical reasoning with two objectives: answer correctness as the *primary* reward and reasoning efficiency as the *auxiliary* reward. We train on DAPO-17K (Yu et al., 2026) and evaluate on OlympiadBench (He et al., 2024), Minerva Math (Lewkowycz et al., 2022), AMC (Li et al., 2024), MATH-500 (Hendrycks et al., 2021), AIME 2024, and AIME 2025. For a response y , the correctness reward is $R_{\text{acc}}(y) = \mathbf{1}\{\hat{y} = y^*\}$, where $\hat{y}$ is the extracted final answer and y^* is the reference answer. We define the efficiency reward as $R_{\text{len}}(y) = \mathbf{1}\{L(y) \leq T\}$, where $L(y)$ is the number of generated response tokens and $T = 8,192$.

Table 1: **Mathematical reasoning results.** Δ rows report changes relative to the Base model. Higher is better for both accuracy (Acc.) and length compliance (Len.). The best result within each optimization family is in **bold**.

Benchmark	Base		PPO-based				GRPO-based					
	Base		PPO		PAPO		GRPO		GDPO		PAPO	
	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.
Olympiad	38.8	51.6	35.9	77.4	42.9	87.2	38.9	75.7	39.6	74.1	43.2	90.2
Δ Olympiad	—	—	-2.9	+25.8	+4.1	+35.6	+0.1	+24.1	+0.8	+22.5	+4.4	+38.6
Minerva	16.7	95.8	16.7	97.9	18.8	97.9	16.7	97.9	18.8	97.9	20.8	100.0
Δ Minerva	—	—	0.0	+2.1	+2.1	+2.1	0.0	+2.1	+2.1	+2.1	+4.1	+4.2
AMC	82.5	60.0	77.5	92.5	95.0	85.0	82.5	97.5	85.0	87.5	92.5	87.5
Δ AMC	—	—	-5.0	+32.5	+12.5	+25.0	0.0	+37.5	+2.5	+27.5	+10.0	+27.5
MATH	85.4	83.0	85.0	97.0	89.6	94.8	84.4	92.8	85.2	92.6	89.8	97.8
Δ MATH	—	—	-0.4	+14.0	+4.2	+11.8	-1.0	+9.8	-0.2	+9.6	+4.4	+14.8
AIME24	56.7	33.3	23.3	43.3	53.3	70.0	46.7	63.3	43.3	60.0	70.0	76.7
Δ AIME24	—	—	-33.4	+10.0	-3.4	+36.7	-10.0	+30.0	-13.4	+26.7	+13.3	+43.4
AIME25	40.0	16.7	23.3	53.3	40.0	46.7	36.7	60.0	33.3	53.3	43.3	73.3
Δ AIME25	—	—	-16.7	+36.6	0.0	+30.0	-3.3	+43.3	-6.7	+36.6	+3.3	+56.6
Overall	57.6	64.1	54.5	83.6	61.1	90.2	56.9	82.7	57.3	81.6	62.1	93.0
Δ Overall	—	—	-3.1	+19.5	+3.5	+26.1	-0.7	+18.6	-0.3	+17.5	+4.5	+28.9

Table 2: **Agentic tool-use results.** Δ rows report changes relative to the Base model. Higher is better for accuracy (Acc.), while lower is better for tool calls. The best result within each optimization family is in **bold**.

Difficulty	Base		PPO-based				GRPO-based					
	Base		PPO		PAPO		GRPO		GDPO		PAPO	
	Acc.	Calls	Acc.	Calls	Acc.	Calls	Acc.	Calls	Acc.	Calls	Acc.	Calls
Easy	88.11	1.33	93.78	0.15	94.56	0.22	93.89	0.13	93.89	0.08	95.67	0.12
Δ Easy	—	—	+5.67	-1.18	+6.45	-1.11	+5.78	-1.20	+5.78	-1.25	+7.56	-1.21
Medium	88.89	1.41	87.22	0.23	92.44	0.49	87.00	0.16	89.77	0.14	89.22	0.15
Δ Medium	—	—	-1.67	-1.18	+3.55	-0.92	-1.89	-1.25	+0.88	-1.27	+0.33	-1.26
Hard	75.67	1.47	77.78	0.73	79.56	0.96	82.22	0.69	76.89	0.52	84.89	0.71
Δ Hard	—	—	+2.11	-0.74	+3.89	-0.51	+6.55	-0.78	+1.22	-0.95	+9.22	-0.76
Overall	84.19	1.40	86.25	0.37	88.25	0.56	87.70	0.33	86.85	0.26	89.93	0.33
Δ Overall	—	—	+2.06	-1.03	+4.06	-0.84	+3.51	-1.07	+2.66	-1.14	+5.74	-1.07

We set $R_{\text{pri}} = R_{\text{acc}}$ and $R_{\text{aux}} = R_{\text{len}}$. We report answer accuracy and length-budget compliance, with accuracy serving as the primary evaluation metric.

Results. Table 1 summarizes the results on mathematical reasoning. PAPO consistently improves the primary objective under both PPO- and GRPO-based training. On average, PAPO raises accuracy from 54.5 to 61.1 over PPO and from 56.9 to 62.1 over GRPO, while also improving length-budget compliance from 83.6 to 90.2 and from 82.7 to 93.0, respectively. Compared with GDPO, PAPO achieves higher average accuracy, improving from 57.3 to 62.1, together with stronger length compliance, increasing from 81.6 to 93.0. The gains are particularly pronounced on the more challenging Olympiad and AIME benchmarks, suggesting that preserving primary-objective priority becomes especially important when correctness and efficiency are in stronger conflict.

6.2 AGENTIC TOOL USING

Data and Reward Settings. We evaluate PAPO on WHEN2TOOL (Sun et al., 2026), which contains 18 tool-use environments spanning single-hop and multi-hop tasks with easy, medium, and hard instances. We use the official training and test splits with 1,080 and 2,700 examples respectively to train Qwen3-8B (Yang et al., 2025). We treat answer correctness as the *primary* reward and tool-use efficiency as the *auxiliary* reward. For a response y , the primary reward is $R_{\text{pri}}(y) = \mathbb{1}\{\text{Ans}(y) = \text{Ans}^*\}$, where Ans^* is the reference answer. Let N_{call} denote the number of executed tool calls and B the task-specific tool budget; we define the auxiliary reward as $R_{\text{aux}} = \max(0, 1 -$

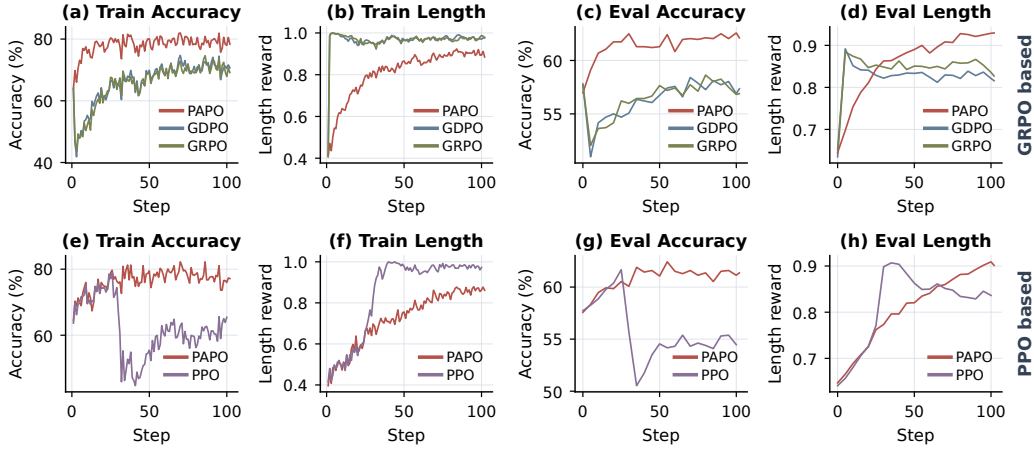


Figure 3: **PAPO Dynamics.** PAPO maintains stronger and more stable accuracy while progressively improving the auxiliary objective for GRPO-based (top) and PPO-based (bottom) methods.

$N_{\text{call}}/(B + \delta)$ with $\delta = 1$. We set B from task annotations, using the number of annotated steps when available, $B = 3$ for multi-step examples without explicit steps, and $B = 1$ otherwise. The efficiency reward is not conditioned on correctness, allowing genuine conflicts between task success and tool-use efficiency. We report accuracy and the average number of tool calls, both overall and across difficulty levels.

Results. Table 2 reports the results on agentic tool use. PAPO improves overall accuracy from 86.25 to 88.25 over PPO and from 87.70 to 89.93 over GRPO. Under GRPO, this gain is achieved while maintaining the same average number of tool calls at 0.33, indicating that the improvement does not rely on additional tool usage. Under PPO, PAPO uses slightly more tools than the PPO baseline, with 0.56 calls on average compared with 0.37, but achieves higher task accuracy. This suggests that aggressively minimizing tool usage can sacrifice task success. Compared with GDPO, PAPO further improves overall accuracy from 86.85 to 89.93 while retaining similarly efficient tool use. These results suggest that PAPO avoids indiscriminately suppressing auxiliary behavior and instead preserves tool use when it remains beneficial for successful task completion.

6.3 TRAINING AND EVALUATION DYNAMICS

To answer **RQ2**, we further illustrate how PAPO balances correctness and reasoning efficiency throughout training in Figure 3. Across both GRPO- and PPO-based settings, baseline methods often optimize the auxiliary length objective rapidly without consistently improving evaluation accuracy, and this effect is particularly pronounced for PPO, where aggressive length optimization coincides with unstable or degraded correctness. In contrast, PAPO maintains stronger and more stable accuracy while progressively improving the length reward, ultimately achieving a better balance between the two objectives. These dynamics support our motivation that an easily optimized auxiliary objective can interfere with the primary task under unconstrained aggregation, whereas PAPO allows auxiliary supervision to refine the policy without overriding correctness.

6.4 ABLATION STUDIES

To answer **RQ3**, we conduct ablations on the multi-head critic and the auxiliary coefficient α to isolate the contribution of each component. **(a) Ablation on α .** Figure 4 studies the effect of the auxiliary coefficient α . Simply tuning α under conventional weighted aggregation does not provide reliable control over objective priority, as optimization can still favor the objective that is easier to improve rather than following the intended weighting (Liu et al., 2026a). In contrast, PAPO consistently maintains stronger primary-task performance across different values of α , while retaining competitive auxiliary performance. **(b) Ablation on Multi-Head Critic.** Table 3 disentangles the contributions of the multi-head critic (MHC) and PAPO under PPO-based training. Introducing MHC alone provides modest improvements, increasing math accuracy from 54.5 to 55.7 and tool-use accuracy from 86.25 to 86.84, together with improved length compliance. Adding PAPO on top

Table 3: PPO component ablation. MHC denotes the multi-head critic. Best results are in **bold** and second-best results are underlined.

Method	Math		Tool Use	
	Acc. $\uparrow$	Len. $\uparrow$	Acc. $\uparrow$	Calls $\downarrow$
Base	57.6	64.1	84.19	1.40
PPO	54.5	83.6	86.25	0.37
PPO + MHC	55.7	<u>87.2</u>	86.84	<u>0.41</u>
PPO + MHC + PAPO	61.1	90.2	88.25	0.56

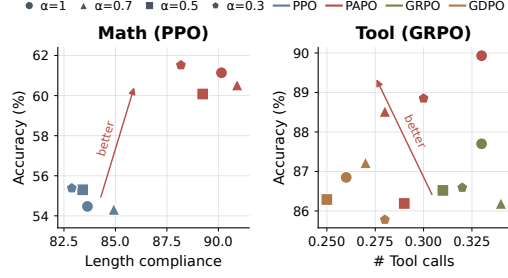


Figure 4: Sensitivity to the auxiliary weight α .

Table 4: **Math results on DeepSeek-R1-Distill-1.5B**. Higher is better for both accuracy (Acc.) and length compliance (Len.). Best results are in **bold** and second-best results are underlined.

Method	Olympiad		Minerva		AMC		MATH		AIME24		AIME25		Overall	
	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.	Acc.	Len.
Base	22.8	43.1	15.6	81.3	47.5	50.0	66.2	75.8	16.7	16.7	16.7	13.3	39.4	60.0
PPO	<u>28.6</u>	<u>66.1</u>	16.7	<u>93.8</u>	65.0	80.0	<u>73.6</u>	<u>91.0</u>	16.7	<u>36.7</u>	20.0	46.7	<u>45.8</u>	<u>75.8</u>
PAPO	29.9	74.6	16.7	95.8	<u>62.5</u>	80.0	78.0	91.8	16.7	43.3	20.0	46.7	48.1	80.7

of MHC yields substantially larger gains, raising math accuracy to 61.1 and tool-use accuracy to 88.25. These results indicate that both components contribute to PAPO’s improvement.

6.5 ADDITIONAL MODEL SCALE, REWARD AND DOMAINS

To answer **RQ4**, we extend PAPO to additional models, scales, and different reward semantics and domains, including multimodal safety alignment.

Additional Model Scale To examine whether PAPO generalizes beyond the primary model setting, we further evaluate it on additional smaller-scale backbone. Table 4 further demonstrates the effectiveness of PAPO on DeepSeek-R1-Distill-1.5B (Guo et al., 2025). Compared with PPO, PAPO improves average accuracy from 45.8 to 48.1 and length compliance from 75.8 to 80.7, with consistent gains across most benchmarks. These results indicate that PAPO remains effective across different model families and reasoning backbones while jointly improving the primary and auxiliary objectives. We also provide results of Qwen3-1.7B for agentic tool use in Section A.1.

Additional Domain We further evaluate PAPO in the multimodal safety-alignment setting of SafeRLHF-V (Ji et al., 2025), treating *safety* as the primary objective and *helpfulness* as the auxiliary objective. We evaluate Qwen2-VL-7B (in Section A.1) and LLaVA-1.6-7B (Liu et al., 2024a) on BeaverTails-V (Ji et al., 2025) and MM-SafetyBench (Liu et al., 2024b) against Base, RLHF (Ouyang et al., 2022) RLHF-V (Sun et al., 2024), LLaVA-RLHF (Sun et al., 2024), and SafeRLHF-V (Ji et al., 2025). Figure 5 shows that PAPO consistently improves safety while preserving strong helpfulness, and achieves a favorable trade-off across both model families. This further demonstrates that PAPO generalizes to different reward semantics and domains beyond reasoning and tool use.

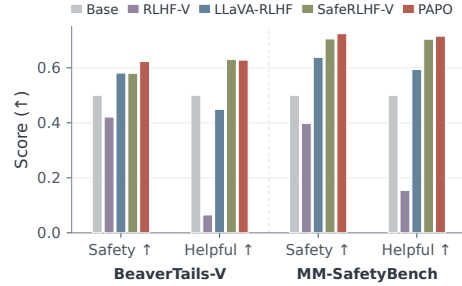


Figure 5: Multimodal safety alignment.

7 CONCLUSION

We study multi-reward reinforcement learning with asymmetric objective roles and identify *advantage priority inversion*, where auxiliary signals can override the preference induced by a primary objective. We propose **Primary Advantage Policy Optimization (PAPO)**, a priority-aware advantage aggregation framework that preserves aligned auxiliary supervision while smoothly constraining conflicting signals. PAPO can be instantiated in both GRPO and PPO, including token-level optimization with objective-specific critics. Across mathematical reasoning, agentic tool use, and

multimodal safety alignment, PAPO consistently improves primary-objective performance while maintaining strong auxiliary behavior, with gains that generalize across models, scales, reward semantics, and optimization frameworks. These results suggest that explicitly modeling objective priority is an effective principle for multi-reward policy optimization.

REFERENCES

- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. In *Second Conference on Language Modeling*, 2025.
- Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773*, 2023.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Yuki Ichihara and Yuu Jinnai. Auto-weighted group relative preference optimization for multi-objective text generation tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1134–1147. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-industry.80.
- Jiaming Ji, Xinyu Chen, Rui Pan, Han Zhu, Jiahao Li, Donghai Hong, Boyuan Chen, Jiayi Zhou, Kaile Wang, Juntao Dai, Chi-Min Chan, Sirui Han, Yike Guo, and Yaodong Yang. Safe rlhf-v: Safe reinforcement learning from multi-modal human feedback. In *Advances in Neural Information Processing Systems*, 2025.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 2022.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 2024.
- Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024a. URL <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, et al. Dler: Doing length penalty right-incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization. *arXiv preprint arXiv:2601.05242*, 2026a.

- Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. Learn to reason efficiently with adaptive length-based reward shaping. In *International Conference on Learning Representations*, 2026b.
- Xin Liu, Yichen Zhu, Jindong Gu, Yunshi Lan, Chao Yang, and Yu Qiao. Mm-safetybench: A benchmark for safety evaluation of multimodal large language models. In *European Conference on Computer Vision*, pp. 386–403. Springer, 2024b.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 2022.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Chung-En Sun, Linbo Liu, Ge Yan, Zimo Wang, and Tsui-Wei Weng. Llm agents already know when to call tools – even without reasoning. *arXiv preprint arXiv:2605.09252*, 2026.
- Zhiqing Sun, Sheng Shen, Shengcao Cao, Haotian Liu, Chunyuan Li, Yikang Shen, Chuang Gan, Liangyan Gui, Yu-Xiong Wang, Yiming Yang, et al. Aligning large multimodal models with factually augmented rlhf. In *Findings of the association for computational linguistics: ACL 2024*, 2024.
- Microsoft AI Team et al. Mai-thinking-1: Building a hill-climbing machine. Technical report, Technical report, Microsoft AI, 2026. [https://microsoft.ai/pdf/mai-thinking ...](https://microsoft.ai/pdf/mai-thinking...), 2025.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- Wen Wang, Jiahua Bao, Yongsiqu Tu, Yihao Liu, Haotian Zhou, Haoxuan Ma, Mengyu Zhou, Wenkui Fan, Junwei He, Xiaoxi Jiang, and Guanjun Jiang. Smopd: Multi-reward reinforcement learning via specialize-and-merge online policy distillation. *arXiv preprint arXiv:2608.03092*, 2026.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 2026.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A ADDITIONAL EXPERIMENTAL RESULTS

A.1 ADDITIONAL MODELS' RESULTS

Additional Tool-Use Results. Table 5 shows the tool-use results of applying PAPO and baselines to the Qwen3-1.7B (Yang et al., 2025) model. PAPO achieves the highest overall accuracy of 81.11, compared with 75.96 for GRPO and 76.85 for GDPO, with notable gains on the medium and hard subsets. Although PAPO uses slightly more tool calls on average, the higher accuracy suggests that it avoids over-optimizing tool efficiency at the cost of task success with comparable tool call efficiency.

Table 5: **Tool-use results on Qwen3-1.7B.** Higher is better for accuracy, while lower is better for tool calls. Best results are in **bold** and second-best results are underlined.

Method	Easy		Medium		Hard		Overall	
	Acc. ↑	Calls ↓	Acc. ↑	Calls ↓	Acc. ↑	Calls ↓	Acc. ↑	Calls ↓
Base	60.11	1.33	60.56	1.43	55.89	1.49	58.85	1.42
GRPO	85.33	0.18	76.33	0.30	66.22	0.65	75.96	0.38
GDPO	88.86	0.13	<u>76.00</u>	0.21	<u>66.00</u>	0.54	<u>76.85</u>	0.29
PAPO	<u>88.00</u>	<u>0.17</u>	80.78	<u>0.29</u>	74.56	<u>0.87</u>	81.11	<u>0.44</u>

Additional Safety Alignment Results. We continue to conduct additional safety alignment experiments on the additional multi-modality model Qwen2-VL-7B (Wang et al., 2024) and illustrate the results in Figure 6. Compared with other baselines, our PAPO achieves the best primary safety performance while having the best or very competitive auxiliary helpful score.

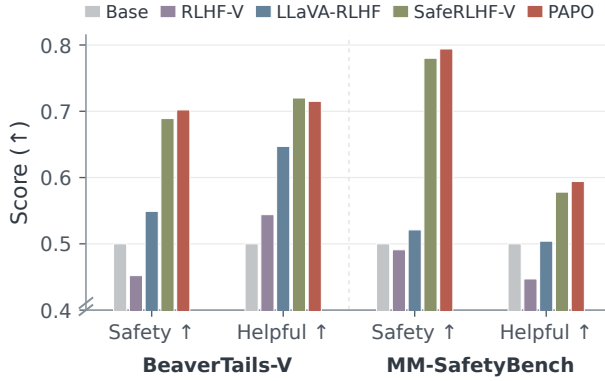


Figure 6: Multimodal safety alignment.

A.2 TRAINING AND EVALUATION DYNAMICS FOR TOOL

Figure 7 shows the optimization dynamics of task correctness and tool-use efficiency. Across both GRPO- and PPO-based methods, the number of tool calls decreases rapidly, suggesting that the auxiliary efficiency objective is relatively easy to optimize; however, lower tool usage does not necessarily yield a better policy. PAPO maintains stronger and more stable evaluation accuracy while using a moderate number of tool calls, whereas GRPO, GDPO, and especially standard PPO can reduce tool usage more aggressively at the cost of weaker or more volatile correctness.

A.3 PER-DATASET EVALUATION DYNAMICS

To complement the aggregated training dynamics in the main text, we further report evaluation dynamics during training separately for each benchmark (math) and difficulty level (tool using).

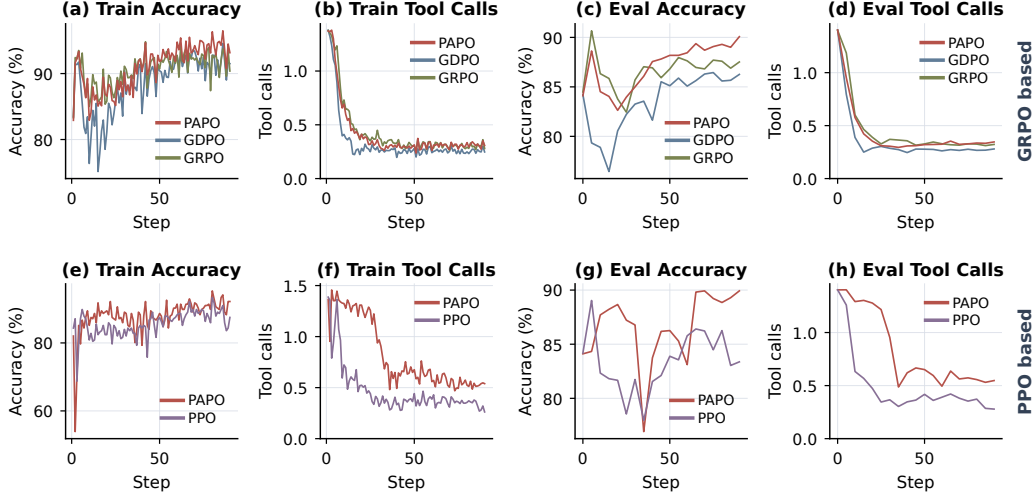


Figure 7: **Dynamics on agentic tool use.** Although baseline methods can reduce tool calls aggressively, this behavior can coincide with degraded accuracy. PAPO maintains stronger primary-objective performance while still reducing tool usage, yielding a better trade-off for GRPO-based (top) and PPO-based (bottom) methods.

These results provide a more fine-grained view of how PAPO balances the primary and auxiliary objectives throughout optimization.

Mathematical Reasoning. Figure 8 presents the per-dataset evaluation dynamics on AIME24, AIME25, AMC, MATH-500, Minerva, and Olympiad. Across both optimization families, the baselines tend to improve the auxiliary length objective relatively quickly, but this does not consistently bring stronger correctness. Under GRPO-based optimization, GRPO and GDPO often reach comparable or higher length compliance early in training while remaining substantially below PAPO in accuracy, particularly on AIME24, AIME25, AMC, MATH-500, and Olympiad. PAPO instead maintains stronger accuracy while progressively improving length compliance, often matching or surpassing the baselines toward the end of training. The same pattern is even more pronounced for PPO: PPO rapidly increases the length reward, but this is accompanied by large accuracy drops on evaluation benchmarks, whereas PAPO has a more gradual efficiency improvement and preserves very stable correctness improvements. Minerva is less discriminative because length compliance quickly approaches its ceiling and accuracy varies over a narrow discrete range. Overall, the per-dataset trajectories show that GRPO- and PPO-based baselines aggressively optimizing the auxiliary length objective, interfering with correctness, while PAPO better preserves the primary accuracy objective as efficiency improves.

Agentic Tool Use. Figure 9 reports evaluation dynamics separately for easy, medium, and hard tool-use subsets. All methods rapidly reduce tool calls, indicating that the auxiliary efficiency objective is comparatively easy to optimize. However, the difference in accuracy becomes increasingly apparent as task difficulty grows. PAPO maintains strong accuracy across all three difficulty levels and is particularly robust on medium and hard subsets, where more aggressive reductions in tool usage can coincide with unstable or lower accuracy. This effect is especially visible in the PPO-based setting, where PPO often reaches fewer tool calls earlier but exhibits considerably more volatile correctness. PAPO instead retains additional tool usage when needed, achieving our intended priority: tool efficiency should refine the solution process without overriding successful task completion.

B PRIORITY PRESERVATION UNDER POST-FUSION NORMALIZATION

PAPO is designed to prevent an auxiliary objective from reversing the preference induced by the primary objective during advantage fusion. In practice, however, the fused advantages may be further

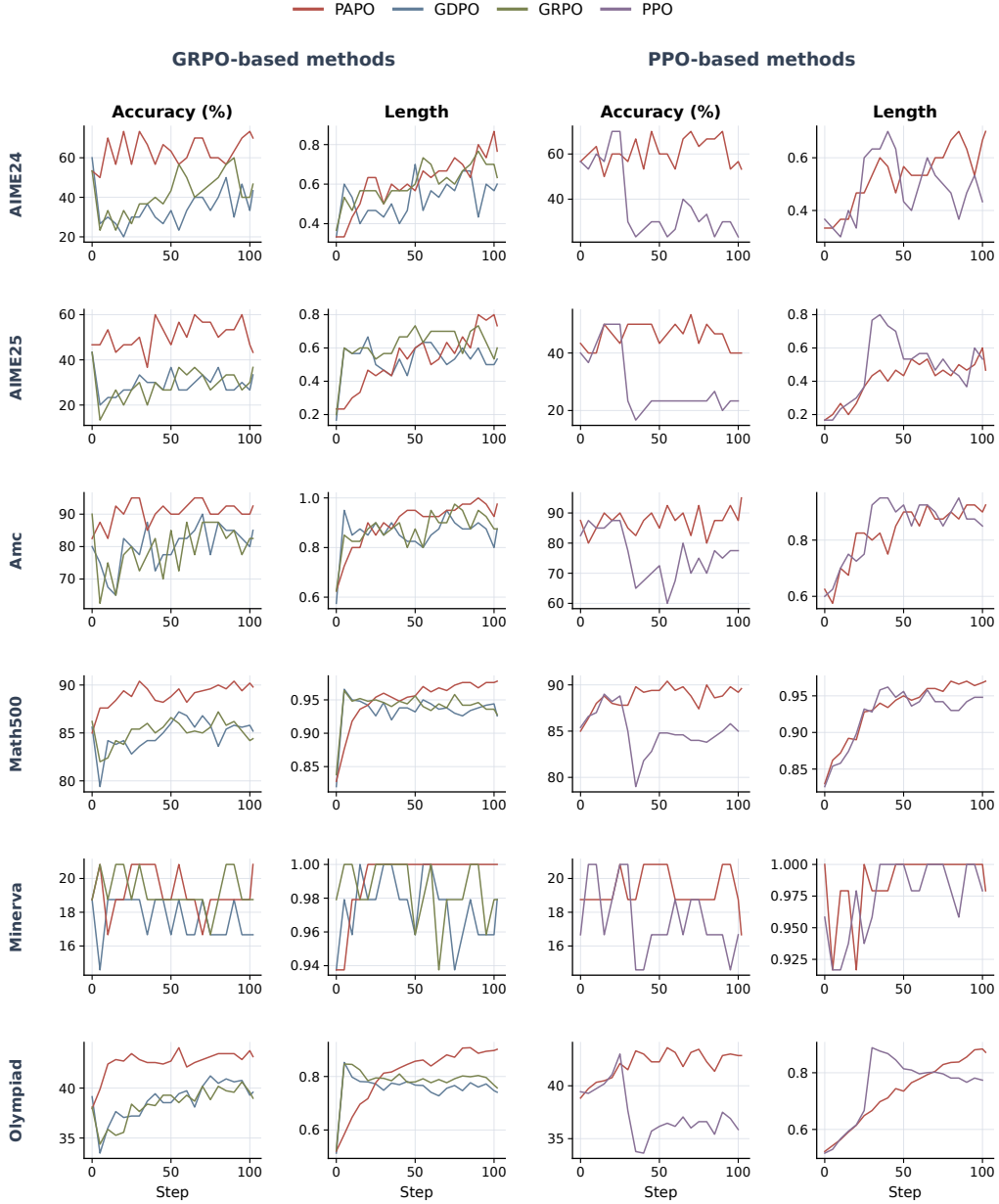


Figure 8: **Per-dataset evaluation dynamics on mathematical reasoning.** We report accuracy and length compliance throughout training for each benchmark under GRPO- and PPO-based optimization.

normalized before being passed to the policy-gradient objective. We clarify here which properties are preserved by PAPO before and after this normalization.

Sign preservation at the fusion stage. Consider a nonzero primary advantage A_{pri} and an auxiliary advantage A_{aux} . When the two advantages are aligned, PAPO reduces to additive aggregation and therefore trivially preserves the sign of the primary advantage. When they conflict, the PAPO update can be written as

$$A_{\text{agg}} = A_{\text{pri}} \left[1 - \tanh \left(\frac{\alpha |A_{\text{aux}}|}{|A_{\text{pri}}| + \epsilon} \right) \right], \quad (9)$$

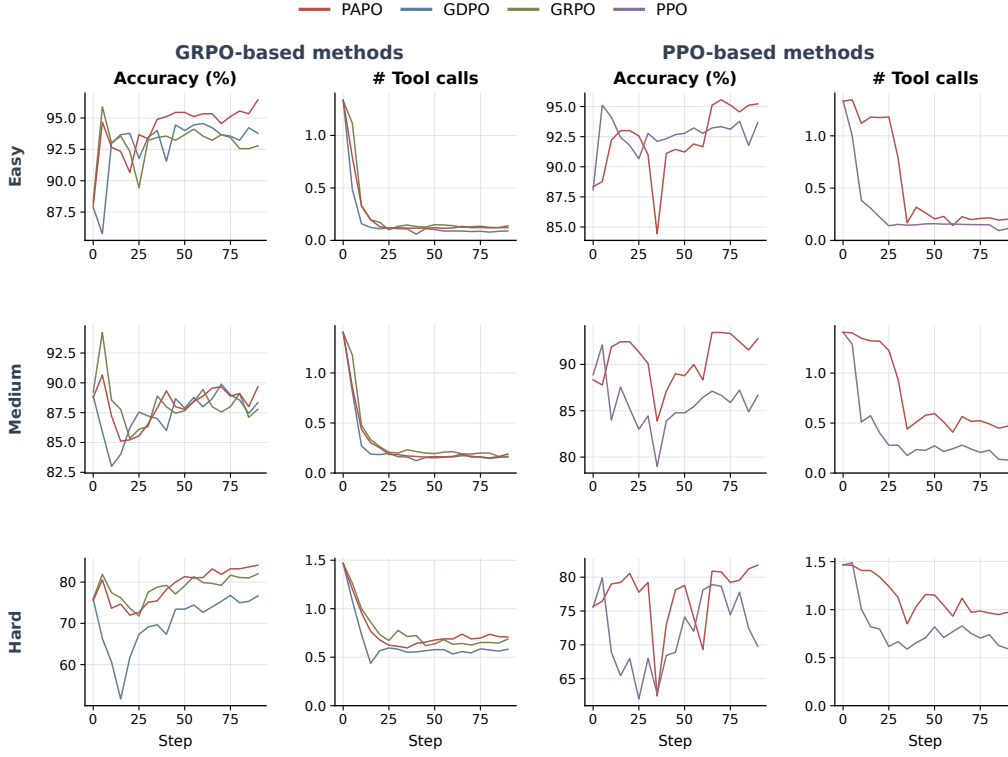


Figure 9: **Per-difficulty evaluation dynamics on agentic tool use.** We report accuracy and the number of tool calls for easy, medium, and hard instances under GRPO- and PPO-based optimization.

where the numerically stabilized denominator is chosen to preserve the sign of A_{pri} . Since

$$0 \leq \tanh\left(\frac{\alpha|A_{\text{aux}}|}{|A_{\text{pri}}| + \epsilon}\right) < 1, \quad (10)$$

the multiplicative term in Eq. 9 is strictly positive. Therefore,

$$\text{sign}(A_{\text{agg}}) = \text{sign}(A_{\text{pri}}). \quad (11)$$

Thus, before normalization, an auxiliary signal can reduce the magnitude of a conflicting primary update, but cannot reverse its direction.

Primary-consistent ordering. The sign-preservation property immediately induces an ordering between samples favored and disfavored by the primary objective.

For any two advantages i and j such that $A_{i,\text{pri}} > 0 > A_{j,\text{pri}}$, PAPO guarantees

$$A_{i,\text{agg}} > A_{j,\text{agg}}. \quad (12)$$

This property is weaker than preserving the complete ranking induced by A_{pri} , which is neither required nor desirable. Auxiliary objectives are still allowed to refine the relative preference among samples that lie on the same side of the primary decision boundary. Instead, Proposition B guarantees that auxiliary signals cannot move a primary-disfavored sample above a primary-favored one. We refer to this property as *primary-consistent ordering*.

Effect of batch or group normalization. For PPO, we normalize the fused advantages within a mini-batch, while GRPO applies the analogous operation within a rollout group:

$$\bar{A}_i = \frac{A_{i,\text{agg}} - \mu}{\sigma + \epsilon}. \quad (13)$$

Because $\sigma + \epsilon > 0$, Eq. 13 is a strictly increasing affine transformation. Hence, for any pair i, j belonging to the same normalization set,

$$A_{i,\text{agg}} > A_{j,\text{agg}} \implies \bar{A}_i > \bar{A}_j. \quad (14)$$

Consequently, the primary-consistent ordering established by PAPO is preserved after normalization.

Importantly, subtracting the batch or group mean can change the absolute sign of an individual advantage. For example, a small positive fused advantage may become negative if it lies below the batch mean. This does not correspond to an auxiliary-induced priority inversion: the zero point has been shifted uniformly for all samples in the normalization set, while their pairwise ordering remains unchanged. Our guarantee therefore concerns the *relative preference structure* established by PAPO, rather than invariance of every individual advantage sign after centering.

Interpretation. This distinction is natural for advantage-based policy optimization. Advantages are inherently defined relative to a baseline, and subsequent centering changes this reference point. PAPO controls a different source of interference: whether an auxiliary objective can overturn the preference induced by the primary objective during multi-reward fusion. The resulting optimization can therefore be viewed hierarchically: the primary objective determines the coarse preference between favored and disfavored samples, while auxiliary objectives provide finer-grained supervision within this structure.

Strict sign-preserving normalization. If preserving the absolute sign of the fused advantage through the final actor update is desired, PAPO can alternatively use scale-only normalization,

$$\bar{A}_i = \frac{A_{i,\text{agg}}}{\sigma + \epsilon}. \quad (15)$$

Since the denominator is strictly positive, this variant preserves both Eq. 11 and the pairwise ordering. In our default implementation, we retain standard centered normalization for compatibility with conventional PPO and GRPO training, and interpret PAPO’s priority guarantee in terms of the primary-consistent ordering described above.

C TRAINING AND IMPLEMENTATION DETAILS

We provide the training configurations used in our mathematical reasoning and agentic tool-use experiments. Within each optimization family, all methods use the same data, rollout configuration, training budget, and optimization hyperparameters unless otherwise specified. PAPO only changes how the objective-specific advantages are aggregated and, for PPO, the corresponding critic parameterization. All experiments are conducted on a single node with 8 GPUs.

C.1 MATHEMATICAL REASONING

GRPO-based methods. For the GRPO-based mathematical reasoning experiments (GRPO, GDPO, and PAPO-GRPO), we train on DAPO-17K and evaluate on the mathematical reasoning benchmarks described in the main paper. The maximum prompt and response lengths are 1,024 and 12,000 tokens, respectively. The length-reward threshold is fixed to 8,000 response tokens. For each prompt, we sample $G = 4$ responses. We use a global training batch size of 512, a policy mini-batch size of 64, and a micro-batch size of 8 per GPU. The actor learning rate is 1×10^{-5} , and all models are trained for 3 epochs. We use asymmetric PPO clipping with lower and upper clipping coefficients of 0.20 and 0.28, respectively. The policy objective includes an MSE-style KL regularization term with coefficient 5×10^{-4} , while KL is not incorporated into the environmental reward. For standard GRPO, the scalar training reward is the sum of correctness and length rewards with equal weights:

$$R = R_{\text{acc}} + R_{\text{len}}. \quad (16)$$

For GDPO, we follow the implementations in Liu et al. (2026a) to make correctness and length retained as separate reward dimensions, independently normalized within each rollout group. For PAPO-GRPO, correctness is treated as the primary objective and length as the auxiliary objective. Unless otherwise specified, we use an auxiliary coefficient $\alpha = 1.0$. Rollout groups without effective reward variation are filtered during GRPO-based training. Other detailed hyperparameters are listed in Table 6.

PPO-based methods. The PPO and PAPO-PPO experiments use the same training data, sequence-length limits, batch sizes, learning rate, clipping range, KL regularization, and 3-epoch training budget as the GRPO-based experiments. The main difference is that PPO uses one rollout per prompt ($G = 1$) and introduces a learned critic. The critic learning rate is 1×10^{-5} with no critic warm-up. PPO uses a single value head trained on the scalar return associated with

$$R = R_{\text{acc}} + R_{\text{len}}. \quad (17)$$

PAPO-PPO instead uses the dual-head critic introduced in the main paper: one value head estimates the correctness return and the other estimates the length return. The two critic losses are equally weighted, which only determines the relative contribution of the two value-function losses and is independent of the auxiliary-advantage coefficient used by the actor. We use $\alpha = 1.0$ for PAPO-PPO unless otherwise specified. Other detailed hyperparameters are listed in Table 6.

Table 6: Training hyperparameters for mathematical reasoning experiments. Settings shared across methods are held fixed within each optimization family.

Hyperparameter	GRPO-based	PPO-based
Training batch size	512	512
Policy mini-batch size	64	64
Micro-batch / GPU	8	8
Rollouts / prompt	4	1
Maximum prompt length	1,024	1,024
Maximum response length	12,000	12,000
Length-reward threshold	8,000	8,000
Actor learning rate	1×10^{-5}	1×10^{-5}
Critic learning rate	–	1×10^{-5}
Training epochs	3	3
PPO clipping range	[0.20, 0.28]	[0.20, 0.28]
Actor KL coefficient	5×10^{-4}	5×10^{-4}
KL type	MSE	MSE
PAPO auxiliary coefficient α	1.0	1.0

C.2 AGENTIC TOOL USE

GRPO-based methods. For the When2Tool experiments, the training and evaluation splits contain 1,080 and 2,700 instances, respectively, combining both single-hop and multi-hop tasks. The maximum prompt length is 4,096 tokens and the maximum generated response length is 2,048 tokens. We sample $G = 8$ trajectories per prompt as group rollouts. We use a global training batch size of 256, a policy mini-batch size of 32, and a micro-batch size of 8 per GPU. The actor learning rate is 1×10^{-5} , and models are trained for 20 epochs. We do not apply either reward-level KL penalties or actor-level KL regularization in the GRPO-based tool-use experiments. During training, the agent is allowed at most 5 assistant turns and 4 user/environment turns, with a maximum tool-response length of 2,048 tokens. We allow at most one parallel tool call and disable explicit thinking mode during rollout. As described in Section 6.2, answer correctness is the primary reward and tool-use efficiency is the auxiliary reward. Standard GRPO optimizes the scalar reward

$$R = R_{\text{pri}} + R_{\text{aux}}. \quad (18)$$

GDPO retains the two reward dimensions separately and combines their normalized advantages with equal weights. For PAPO-GRPO, we use correctness as the primary objective and tool efficiency as the auxiliary objective with $\alpha = 1$. The tool-budget offset in the efficiency reward is fixed to $\delta = 1$. Other detailed hyperparameters are listed in Table 7.

PPO-based methods. PPO-based tool-use experiments use the same dataset, batch configuration, sequence lengths, multi-turn interaction limits, and 40-epoch training budget as the GRPO-based experiments. PPO samples one trajectory per prompt and uses an actor and learned critic with learning rates of 1×10^{-5} for both. We use asymmetric PPO clipping with lower and upper coefficients of 0.20 and 0.28, respectively. Standard PPO optimizes the scalar combination

$$R = R_{\text{pri}} + R_{\text{aux}}, \quad (19)$$

whereas PAPO-PPO maintains the two reward dimensions separately and applies PAPO to their corresponding advantages. PAPO-PPO uses the dual-head critic, with one value head for answer correctness and one for tool efficiency. We set the auxiliary coefficient to $\alpha = 0.3$ for PAPO-PPO. As in the GRPO-based setting, the tool-budget offset is $\delta = 1$. Other detailed hyperparameters are listed in Table 7.

Table 7: Training hyperparameters for the When2Tool experiments.

Hyperparameter	GRPO-based	PPO-based
Training batch size	256	256
Policy mini-batch size	32	32
Micro-batch / GPU	8	8
Rollouts / prompt	8	1
Maximum prompt length	4,096	4,096
Maximum response length	2,048	2,048
Maximum assistant turns	5	5
Maximum environment turns	4	4
Maximum tool-response length	2,048	2,048
Maximum parallel tool calls	1	1
Actor learning rate	1×10^{-5}	1×10^{-5}
Critic learning rate	–	1×10^{-5}
Training epochs	20	20
PPO clipping range	–	[0.20, 0.28]
KL regularization	None	None
Tool-budget offset δ	1.0	1.0
PAPO auxiliary coefficient α	1	0.3

C.3 DATASET DETAILS AND STATISTICS

We provide additional details on the training and evaluation datasets used throughout our experiments. Beyond reporting dataset sizes, we summarize the task composition, difficulty structure, and evaluation scope of each domain. The three settings differ substantially in both modality and objective semantics: mathematical reasoning evaluates correctness under a generation budget, agentic tool use evaluates task success under interaction cost, and multimodal safety alignment evaluates safety jointly with helpfulness.

Competition-level Mathematical Reasoning. We use DAPO-Math-17K (Yu et al., 2026) as the training set for the mathematical reasoning experiments. DAPO-Math-17K contains approximately 17K competition-style mathematical problems paired with verifiable reference answers and was originally constructed for reinforcement-learning post-training. The problems span common competition-mathematics domains such as algebra, geometry, number theory, and combinatorics. Its deterministic answer format makes it suitable for outcome-based RL, as correctness can be evaluated without an external judge model.

We evaluate on six held-out benchmarks that cover substantially different difficulty regimes and sources. AIME 2024 and AIME 2025 contain 30 problems each from the corresponding American Invitational Mathematics Examinations and primarily test difficult high-school competition mathematics. AMC23 contains 40 problems from the 2023 American Mathematics Competitions and represents a relatively easier competition-level regime. MATH-500 is a curated 500-problem subset of MATH (Hendrycks et al., 2021), spanning algebra, geometry, number theory, counting and probability, intermediate algebra, and precalculus. Minerva Math (Lewkowycz et al., 2022) contains 272 quantitative problems drawn primarily from university-level STEM material, including subjects such as differential equations, physics, astronomy, and chemistry. Finally, we use the OE_TO_maths_en_COMP subset of OlympiadBench (He et al., 2024), containing 674 English, text-only, open-ended competition problems drawn from Olympiad-level mathematics. Table 8 summarizes the resulting training and evaluation suite. All six evaluation sets are used only for evaluation and do not contribute to training.

For all datasets, we extract the final answer from the generated response and compare it against the reference answer using the same rule-based verifier. The primary reward is $R_{\text{acc}}(y) = \mathbf{1}\{\hat{y} = y^*\}$.

Table 8: **Datasets used for mathematical reasoning.** DAPO-Math-17K is used for policy optimization, while the remaining datasets are held out for evaluation. The evaluation suite spans standard competition mathematics, Olympiad-level problems, and university-level quantitative reasoning.

Dataset	# Examples	Role	Scope
DAPO-Math-17K	~17K	Train	Competition-style mathematics with verifiable reference answers; covers algebra, geometry, number theory, combinatorics, and related topics.
AMC23	40	Test	2023 AMC competition problems; comparatively short high-school competition mathematics across algebra, geometry, number theory, and combinatorics.
AIME 2024	30	Test	AIME I and II 2024; difficult competition problems with integer final answers.
AIME 2025	30	Test	AIME I and II 2025; difficult competition problems with integer final answers.
MATH-500	500	Test	Curated subset of MATH covering algebra, geometry, number theory, counting/probability, intermediate algebra, and precalculus.
Minerva Math	272	Test	University-level quantitative reasoning from STEM subjects, including mathematics, physics, astronomy, and chemistry.
OlympiadBench	674	Test	English text-only open-ended competition mathematics (OE_TO_maths_en_COMP), emphasizing Olympiad-level reasoning.

The auxiliary objective measures whether the complete reasoning trajectory fits within the generation budget, $R_{\text{len}}(y) = \mathbf{1}\{L(y) \leq T\}$, with $T = 8,192$. Thus, the evaluation suite tests whether improvements in reasoning efficiency can be obtained without sacrificing correctness across substantially different levels of mathematical difficulty.

Agentic Tool Use. We use WHEN2TOOL (Sun et al., 2026) to study multi-reward optimization when the auxiliary objective directly controls interaction cost. Unlike conventional tool-use benchmarks in which every example is constructed to require a tool, WHEN2TOOL explicitly varies whether external tool use is necessary. It contains 18 environments: 15 single-hop environments and three multi-hop environments. The benchmark organizes tool necessity around three forms of model self-assessment: *computational scale*, which asks whether the model can reliably perform the required computation internally; *knowledge boundaries*, which asks whether the required information is available in the model’s parametric knowledge; and *execution reliability*, which asks whether the model can accurately execute a procedural operation without external assistance.

Each category contains five single-hop environments. Computational-scale tasks include arithmetic, statistics, combinatorial counting, matrix operations, and primality testing. Knowledge-boundary tasks cover retrieval, historical facts, game rules, hashing, and decoding. Execution-reliability tasks require manipulating lists, performing date calculations, executing code, resolving schedules, or matching regular expressions. In addition, one three-hop environment is constructed for each category by chaining calculator, retriever, or code-execution operations. Importantly, the easy, medium, and hard splits are not merely generic notions of problem difficulty. They are designed to move the problem across a *tool-necessity boundary*. Easy examples are often directly solvable by the language model, medium examples lie near the boundary where tool use becomes beneficial, and hard examples deliberately increase computational scale, introduce unavailable information, or require sufficiently complex execution that tool use becomes substantially more important.

Table 9 provides the full environment taxonomy and illustrates how difficulty changes within each environment. This controlled construction is particularly useful for our setting because minimizing tool calls is not uniformly desirable. On easy instances, avoiding a redundant call can improve efficiency without affecting correctness, whereas on hard instances an additional call may be essential for successful task completion. We therefore treat correctness as the primary reward and tool efficiency as the auxiliary reward. Specifically, $R_{\text{pri}}(y) = \mathbf{1}\{\text{Ans}(y) = \text{Ans}^*\}$, while

Table 9: **Environment composition of WHEN2TOOL.** Each single-hop environment contains easy, medium, and hard instances. The last column summarizes how each environment moves from tasks that are often solvable internally to tasks for which tool assistance becomes increasingly useful or necessary. The three multi-hop environments contain three dependent operations.

Category	Environment	Capability	Easy → Hard progression
Computational Scale	Calculator	Arithmetic evaluation	Small integer arithmetic → large multi-operation or trillion-scale expressions.
	Statistics	Statistical computation	Mean/median of short lists → variance, standard deviation, or correlation over longer lists.
	Counting	Combinatorial counting	Small combinations such as $\binom{6}{4}$ → large combinatorial quantities such as $\binom{80}{40}$.
	Matrix	Matrix computation	Small 2×2 operations → larger-matrix determinants and other matrix operations.
	Prime	Primality testing	Small familiar integers → large integers whose primality is difficult to determine reliably without computation.
Knowledge Boundaries	Retriever	Information retrieval	Common factual knowledge → obscure or synthetic entities requiring retrieval from the provided corpus.
	Historical Year	Historical fact lookup	Well-known event dates → obscure or synthetic event years unavailable from parametric knowledge.
	Game Rule	Rule/statistic lookup	Familiar game rules → rules or statistics of unfamiliar or synthetic games.
	Hash	String hashing	Familiar hash functions such as MD5 → custom or previously unseen hashing procedures.
	Decoding	String decoding	Familiar encodings such as Morse code → custom ciphers defined externally.
Execution Reliability	List Manipulation	Sequential operations	A few operations on short lists → longer or nested list transformations.
	DateTime	Date computation	Simple calendar operations → date arithmetic across months, years, and leap years.
	Code Executor	Program execution	Simple expressions → loops, recursion, or dynamic-programming programs requiring extended execution.
	Schedule	Constraint scheduling	Simple overlap checks → finding feasible intervals among multiple interacting events.
	Regex Match	Regex execution	Simple patterns on short strings → complex regular expressions over longer inputs.
Multi-hop	Chained Calculator	Dependent computation	Three-step arithmetic chains with progressively larger and more complex intermediate values.
	Chained Retriever	Dependent retrieval	Multi-step retrieval where one retrieved fact determines the next query.
	Chained Code Executor	Dependent execution	Three sequential programs in which each output becomes the input to the next computation.

$R_{\text{aux}} = \max(0, 1 - N_{\text{call}}/(B + \delta))$ with $\delta = 1$. The efficiency reward is intentionally independent of correctness, allowing the two objectives to genuinely conflict.

Multimodal Safety Alignment. For multimodal safety alignment, we follow the setting of SafeRLHF-V (Ji et al., 2025) and use BeaverTails-V for alignment. BeaverTails-V extends preference-based safety supervision to vision-language models and contains paired multimodal prompts and responses with separate annotations for *safety* and *helpfulness*. Each record contains a textual question, an associated image, an image-severity annotation, two candidate model responses, the identity of the more helpful response, binary safety labels for both responses, the safer-response preference, and fine-grained harmlessness and helpfulness scores. The released data are divided into 20 fine-grained safety categories organized under nine broader safety domains, summarized in Table 10. The released BeaverTails-V data contain approximately 30K preference examples for training, while the evaluation portion contains roughly 30 examples per fine-grained category (approximately 590 in total, with small category-level variation). Candidate responses were generated using multiple vision-language models, yielding response pairs that can differ along both safety and helpfulness dimensions. The dataset therefore naturally exposes the non-equivalence of the two objectives: a response can become safer through over-refusal while simultaneously becoming less useful, motivating our choice of safety as the primary objective and helpfulness as the auxiliary objective.

For out-of-distribution safety evaluation, we additionally use MM-SafetyBench (Liu et al., 2024b). MM-SafetyBench contains 5,040 text–image pairs across 13 safety-critical scenarios. In contrast

Table 10: **Safety taxonomy of BeaverTails-V.** The dataset contains 20 fine-grained categories grouped into nine broader safety domains, providing diverse supervision for multimodal safety alignment.

Safety domain	Fine-grained categories
National and public security	Terrorism or extremism; hacking or digital crime.
Health and safety risks	Harmful health content; environmental damage; dangerous behavior.
Physical and psychological harm	Violence and physical harm; psychological harm and manipulation; animal abuse.
Privacy and consent violations	Privacy invasion and surveillance; identity misuse and impersonation.
Discrimination and harassment	Discriminatory depictions; insulting and harassing behavior.
Deceptive and unethical behavior	Deception in personal relationships; financial and academic fraud.
False and sensitive information	False information; sensitive information in key areas.
Disturbing content	Horror and gore; psychological horror and dark themes.
Sexually explicit or inappropriate content	Pornographic content; sexual crimes.

Table 11: **Datasets used for multimodal safety alignment and evaluation.** BeaverTails-V provides safety–helpfulness preference supervision, while MM-SafetyBench provides an independent multimodal safety evaluation.

Dataset	# Examples	Role	Content / Annotation
BeaverTails-V	~30K	Train	Multimodal preference pairs across 20 safety categories; includes separate safety and helpfulness preferences, binary safety labels, image-severity levels, and graded response scores.
BeaverTails-V	~590	Test	Held-out examples distributed across the same 20 fine-grained categories, generally 28–30 examples per category.
MM-SafetyBench	5,040	Test	Text–image safety benchmark covering 13 safety-critical scenarios and multiple image-based attack constructions.

to BeaverTails-V, which provides preference supervision for alignment, MM-SafetyBench is explicitly designed to evaluate whether multimodal models remain safe when potentially harmful intent is conveyed through the interaction between text and images. Its scenarios span illegal activity, hate speech, malware generation, physical harm, economic harm, fraud, sexual content, political lobbying, privacy violations, legal advice, financial advice, health consultation, and government decision making. The benchmark further includes several image construction strategies, including generated images, typography-based images, and combined variants, to test whether harmful instructions can bypass text-only safety mechanisms.

D TRAINING AND REWARD COMPUTATION EXAMPLES

We provide representative examples illustrating the actual training interface and reward computation for mathematical reasoning and agentic tool use. The problem inputs and ground-truth answers are taken from the corresponding training datasets. Since these datasets provide problems and reference answers but do not contain canonical reasoning trajectories, the assistant responses below are valid examples constructed according to the actual training format.

D.1 MATHEMATICAL REASONING EXAMPLE

We first consider a mathematical reasoning example whose ground-truth answer is 34.

Input.

System

You are a helpful math assistant. Solve the problem step by step. Put only the final answer in `\boxed{}` at the end of your response. Do not use an “Answer:” line; the final answer must be extractable from `\boxed{}`.

User

In triangle ABC , $\sin \angle A = \frac{4}{5}$ and $\angle A < 90^\circ$. Let D be a point outside triangle ABC such that $\angle BAD = \angle DAC$ and $\angle BDC = 90^\circ$. Suppose that $AD = 1$ and that $\frac{BD}{CD} = \frac{3}{2}$. If $AB + AC$ can be expressed in the form $\frac{a\sqrt{b}}{c}$ where a, b, c are pairwise relatively prime integers, find $a + b + c$.

The reference answer is

$$y^* = 34.$$

Example response.

Assistant

Since $\sin A = \frac{4}{5}$ and $A < 90^\circ$, we have

$$\cos A = \frac{3}{5}.$$

Because AD bisects $\angle A$, let

$$\theta = \frac{A}{2}.$$

Then

$$\cos \theta = \sqrt{\frac{1 + \cos A}{2}} = \sqrt{\frac{1 + \frac{3}{5}}{2}} = \frac{2}{\sqrt{5}}, \quad \sin \theta = \frac{1}{\sqrt{5}}.$$

Place

$$A = (0, 0), \quad D = (1, 0),$$

with AD on the x -axis, and write

$$AB = m, \quad AC = n.$$

Then

$$B = \left(\frac{2m}{\sqrt{5}}, \frac{m}{\sqrt{5}} \right), \quad C = \left(\frac{2n}{\sqrt{5}}, -\frac{n}{\sqrt{5}} \right).$$

Since $\angle BDC = 90^\circ$,

$$(B - D) \cdot (C - D) = 0,$$

and hence

$$\left(\frac{2m}{\sqrt{5}} - 1 \right) \left(\frac{2n}{\sqrt{5}} - 1 \right) - \frac{mn}{5} = 0.$$

This simplifies to

$$\frac{3}{5}mn - \frac{2}{\sqrt{5}}(m + n) + 1 = 0.$$

Furthermore,

$$BD^2 = m^2 - \frac{4m}{\sqrt{5}} + 1, \quad CD^2 = n^2 - \frac{4n}{\sqrt{5}} + 1.$$

Since

$$\frac{BD}{CD} = \frac{3}{2},$$

we obtain

$$4 \left(m^2 - \frac{4m}{\sqrt{5}} + 1 \right) = 9 \left(n^2 - \frac{4n}{\sqrt{5}} + 1 \right).$$

Solving the two equations gives

$$(m, n) = \left(\frac{\sqrt{5}}{6}, \frac{4\sqrt{5}}{9} \right)$$

or

$$(m, n) = \left(\frac{7\sqrt{5}}{6}, \frac{8\sqrt{5}}{9} \right).$$

The second configuration places D inside triangle ABC , whereas the problem requires D to lie outside. Therefore,

$$AB + AC = \frac{\sqrt{5}}{6} + \frac{4\sqrt{5}}{9} = \frac{11\sqrt{5}}{18}.$$

Thus,

$$a = 11, \quad b = 5, \quad c = 18,$$

and

$$a + b + c = 34.$$

34

Reward computation. The last valid boxed answer is `34`, which matches the ground truth. The response is also shorter than the 8,000-token length threshold. Therefore,

$$R_{\text{acc}} = 1, \quad R_{\text{len}} = 1.$$

For standard GRPO, the two rewards are scalarized with equal weights:

$$R_{\text{score}} = R_{\text{acc}} + R_{\text{len}} = 2.$$

GDPO and PAPO-GRPO instead retain the two reward components separately,

$$(R_{\text{acc}}, R_{\text{len}}) = (1, 1),$$

and compute their objective-specific advantages before aggregation.

D.2 AGENTIC TOOL-USE EXAMPLES

We provide two multi-step tool-use examples that illustrate different forms of agentic dependency. The first requires iterative code execution, where later programs must be instantiated from earlier execution results. The second requires multi-hop retrieval, where later search queries depend on entities obtained in previous retrieval steps.

D.2.1 THREE-STEP CODE EXECUTION EXAMPLE

We first consider a hard multi-step code-execution task. The expected intermediate results are

$$x = 987, \quad y = 549, \quad z = 93,$$

and the final ground-truth answer is

$$y^* = 93.$$

The task-specific tool budget is

$$B = 3.$$

Input.

System

You can use the provided tools when helpful, but tool use is optional. Call at most one tool at a time and use arguments that match its schema. When you have the answer, end with exactly one final answer in the form `\boxed{...}`.

User

Three-step code execution: Step 1: Run the following code. Call the result x.

```
memo = {}
def fib(n):
    if n in memo: return memo[n]
    if n < 2: return n
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
print(fib(16))
```

Step 2: Using x, run: compute sum of all proper divisors of x (excluding x itself). Call the result y.

Step 3: Using y, run: compute min coins for amount=y with coins=[1, 4, 6]. Call the result z.

Return only z.

The instance exposes a single code-execution tool:

Tool

```
{
  "type": "function",
  "function": {
    "name": "run_code",
    "description":
      "Execute a Python code snippet and return its stdout output.",
    "parameters": {
      "type": "object",
```

```

    "properties": {
      "code": {
        "type": "string",
        "description": "Python code to execute."
      },
      "required": ["code"]
    }
  }
}

```

Step 1: computing the Fibonacci value. The first program is directly provided by the user, so the agent executes it:

Assistant

```

<tool_call>
{
  "name": "run_code",
  "arguments": {
    "code": "memo = {} \ndef fib(n): \n    if n in memo: return memo[n] \n    if n < 2: return \n        n \n    memo[n] = fib(n-1) + fib(n-2) \n    return memo[n] \nprint(fib(16)) "
  }
}
</tool_call>

```

The execution environment returns:

Tool

```

{
  "success": true,
  "stdout": "987",
  "stderr": "",
  "exit_code": 0,
  "result": "987"
}

```

The agent therefore obtains

$$x = 987.$$

Step 2: summing the proper divisors. The second program must be constructed using the execution result from the previous step:

Assistant

```

<tool_call>
{
  "name": "run_code",
  "arguments": {
    "code": "x = 987 \ns = 0 \nfor d in range(1, x): \n    if x % d == 0: \n        s += d \n    \nprint(s) "
  }
}
</tool_call>

```

The tool returns:

Tool

```

{
  "success": true,
  "stdout": "549",
  "stderr": "",
  "exit_code": 0,
  "result": "549"
}

```

Hence,

$$y = 549.$$

For reference,

$$987 = 3 \times 7 \times 47,$$

whose proper divisors are

$$1, 3, 7, 21, 47, 141, 329,$$

and therefore

$$1 + 3 + 7 + 21 + 47 + 141 + 329 = 549.$$

Step 3: computing the minimum number of coins. The final program is dynamically instantiated using $y = 549$:

Assistant

```
<tool_call>
{
  "name": "run_code",
  "arguments": {
    "code": "y = 549\ncoins = [1, 4, 6]\ndp = [float('inf')] * (y + 1)\ndp[0] = 0\nfor\n    amount in range(1, y + 1):\n        for coin in coins:\n            if coin <= amount:\n                dp[amount] = min(dp[amount], dp[amount - coin] + 1)\nprint(dp[y])"
  }
}
</tool_call>
```

The execution environment returns:

Tool

```
{
  "success": true,
  "stdout": "93",
  "stderr": "",
  "exit_code": 0,
  "result": "93"
}
```

The agent therefore obtains

$$z = 93.$$

Since the user requests only the final value, the assistant terminates with:

Assistant

93

Reward computation. The final boxed answer matches the ground truth, so

$$R_{\text{pri}} = 1.$$

The trajectory executes three tool calls:

$$N_{\text{call}} = 3.$$

For this multi-step instance,

$$B = 3, \quad \delta = 1.$$

Therefore,

$$R_{\text{aux}} = \max \left(0, 1 - \frac{N_{\text{call}}}{B + \delta} \right) = 1 - \frac{3}{3 + 1} = 0.25.$$

The raw reward vector is

$$(R_{\text{pri}}, R_{\text{aux}}) = (1, 0.25).$$

For standard GRPO, the auxiliary reward coefficient is 0.5, yielding

$$R_{\text{score}} = R_{\text{pri}} + 0.5R_{\text{aux}} = 1 + 0.5 \times 0.25 = 1.125.$$

PAPO-GRPO instead retains the two reward dimensions separately:

$$R_{\text{pri}} = 1, \quad R_{\text{aux}} = 0.25.$$

Their group-relative advantages are normalized independently, after which the auxiliary advantage is weighted by $\alpha = 0.5$ and fused with the primary advantage using the sign-aware PAPO rule.

This example illustrates a genuinely agentic dependency structure. Although the first program is given directly in the prompt, the second program cannot be instantiated until the first execution produces $x = 987$, and the third cannot be instantiated until the second produces $y = 549$. The agent must therefore maintain intermediate execution state and dynamically construct later tool arguments from earlier tool observations.

D.2.2 THREE-HOP RETRIEVAL EXAMPLE

We next consider a three-hop retrieval task from `RetrieverEnv`. The task requires each subsequent query to be constructed from the result of the previous retrieval step.

The dependency chain is

$$\textit{Moby-Dick} \rightarrow \text{Herman Melville} \rightarrow \text{United States} \rightarrow \text{Washington, D.C.},$$

with ground-truth answer

$$y^* = \text{Washington, D.C.}$$

and tool budget

$$B = 3.$$

Input.

System

You can use the provided tools when helpful, but tool use is optional. Call at most one tool at a time and use arguments that match its schema. When you have the answer, end with exactly one final answer in the form `\boxed{...}`.

User

Three-step retrieval: Step 1: Who wrote *Moby-Dick*? Call the answer x . Step 2: In which country was x born? Call the answer y . Step 3: What is the capital of y ? Call the answer z . Return only z .

The instance exposes two local retrieval tools, `search_corpus` and `read_doc`.

Tool

```
[
  {
    "type": "function",
    "function": {
      "name": "search_corpus",
      "description":
        "Search local corpus and return metadata-only hits (id/title/score). Use read_doc to access full text.",
      "parameters": {
        "type": "object",
        "properties": {
          "query": {
            "type": "string",
            "description": "Search query text."
          },
          "top_k": {
            "type": "integer",
            "description": "Maximum number of hits to return."
          }
        },
        "required": ["query"]
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "read_doc",
      "description": "Read a full document by id.",
      "parameters": {
        "type": "object",
        "properties": {
          "doc_id": {
            "type": "string",
```

```

      "description":
        "Document id from search results."
    },
    "required": ["doc_id"]
  }
}
]

```

Step 1: identifying the author. The agent first searches for *Moby-Dick*:

Assistant

```

<tool_call>
{"name": "search_corpus",
 "arguments": {"query": "Moby-Dick", "top_k": 1}}
</tool_call>

```

Tool

```

{
  "success": true,
  "data": {
    "query": "Moby-Dick",
    "total_hits": 10,
    "returned": 1,
    "hits": [
      {
        "id": "doc1",
        "title": "Moby-Dick",
        "snippet":
          "Moby-Dick is an 1851 novel by Herman Melville
          about Captain Ahab's quest to hunt the white whale.",
        "score": 1
      }
    ]
  }
}

```

From this result, the agent obtains

$x = \text{Herman Melville.}$

Step 2: identifying the birth country. The second query is generated from the entity obtained in Step 1:

Assistant

```

<tool_call>
{"name": "search_corpus",
 "arguments": {
   "query": "Herman Melville biography",
   "top_k": 1
 }}
</tool_call>

```

Tool

```

{
  "success": true,
  "data": {
    "query": "Herman Melville biography",
    "total_hits": 10,
    "returned": 1,
    "hits": [
      {
        "id": "doc10",
        "title": "Herman Melville biography",
        "snippet":
          "Herman Melville was born in New York City,

```

```

    United States, in 1819.",
    "score": 3
  }
]
}
}

```

The agent therefore obtains

$$y = \text{United States.}$$

Step 3: identifying the capital. The final query is constructed from the country obtained in Step 2:

Assistant

```

<tool_call>
{"name": "search_corpus",
 "arguments": {
  "query": "United States capital",
  "top_k": 1
 }}
</tool_call>

```

Tool

```

{
  "success": true,
  "data": {
    "query": "United States capital",
    "total_hits": 10,
    "returned": 1,
    "hits": [
      {
        "id": "doc7",
        "title": "United States capital",
        "snippet":
          "The capital of the United States is
          Washington, D.C., established in 1790
          as the seat of the federal government.",
        "score": 3
      }
    ]
  }
}

```

Thus,

$$z = \text{Washington, D.C.}$$

Since the user requests only the final entity, the assistant terminates with:

Assistant

Washington, D.C.

Reward computation. The final boxed answer matches the reference answer, so

$$R_{\text{pri}} = 1.$$

The trajectory contains three tool calls:

$$N_{\text{call}} = 3.$$

For this multi-step instance,

$$B = 3, \quad \delta = 1.$$

Therefore,

$$R_{\text{aux}} = \max \left(0, 1 - \frac{N_{\text{call}}}{B + \delta} \right) = 1 - \frac{3}{3 + 1} = 0.25.$$

The raw reward vector is

$$(R_{\text{pri}}, R_{\text{aux}}) = (1, 0.25).$$

For standard GRPO, the auxiliary reward coefficient is 0.5, giving

$$R_{\text{score}} = R_{\text{pri}} + 0.5R_{\text{aux}} = 1 + 0.5 \times 0.25 = 1.125.$$

PAPo-GRPO does not construct its policy advantage from this scalar score. Instead, it retains the two raw reward dimensions:

$$R_{\text{pri}} = 1, \quad R_{\text{aux}} = 0.25.$$

The corresponding group-relative advantages are normalized independently, after which the auxiliary advantage is weighted by $\alpha = 0.5$ and combined with the primary advantage using the sign-aware PAPo fusion rule.

Multi-hop dependency. This task requires genuine multi-turn state propagation. The second query cannot be instantiated until the first retrieval identifies Herman Melville, and the third query cannot be instantiated until the second retrieval identifies the United States. The agent must therefore maintain the intermediate state

$$x \rightarrow y \rightarrow z,$$

dynamically construct later tool arguments from earlier observations, and respect the one-tool-call-per-turn constraint. The final response, meanwhile, must expose only the requested terminal entity rather than the intermediate retrieval process.

Together, the two tool-use examples illustrate complementary forms of long-horizon agentic behavior. The code-execution example requires dynamically constructing executable programs from intermediate numerical states, whereas the retrieval example requires dynamically constructing semantic queries from intermediate entities. In both cases, task correctness serves as the primary objective, while tool-use efficiency provides auxiliary supervision over how the correct result is obtained.